

DOCUMENTACION TECNICA

PWA DOMIDELIS

Aplicacion Web Progresiva para Gestion de
Domicilios

VERSION
1.0.2

FECHA
Junio 2025

PLATAFORMA
Node.js + PWA

RESUMEN

Documentacion completa de la arquitectura, modulos funcionales, stack tecnologico, Service Workers, autenticacion JWT, notificaciones push, Socket.IO, API endpoints, integracion con Google Apps Script, instalacion, configuracion y despliegue de la plataforma DOMIDELIS.

Contenido

- 1. Introduccion y Vision General 4
 - 1.1 Caracteristicas Principales 4
- 2. Arquitectura del Sistema 5
 - 2.1 Diagrama de Componentes 5
 - 2.2 Flujo de Datos 6
 - 2.3 Estrategia de Desacoplamiento 6
- 3. Stack Tecnologico 6
- 4. Estructura del Proyecto 7
 - 4.1 Directorio Raiz 8
 - 4.2 Directorio frontend/ 8
 - 4.3 Directorio app-tiendas/ 8
 - 4.4 Directorio backend/ 9
- 5. Modulo Frontend - Cliente 9
 - 5.1 Pagina Principal (index.html) 10
 - 5.2 Checkout y Confirmacion 10
- 6. Modulo Frontend - Panel de Administracion 11
 - 6.1 Gestion de Tiendas y Productos 11
 - 6.2 Pedidos Activos y Asignacion 11
 - 6.3 Informes Financieros 11
- 7. Modulo Frontend - Panel del Domiciliario 12

8. Modulo App-Tiendas (PWA Independiente)	12
8.1 Autenticacion de Tiendas	12
8.2 Panel de Productos y Pedidos	13
8.3 Service Worker de Tiendas	13
9. Backend - Servidor Node.js	13
9.1 Configuracion del Servidor	13
9.2 Proxy API Principal	13
9.3 Router de Tiendas	14
9.4 Gestion de Suscripciones Push	14
10. Service Worker y Estrategia de Cache	14
10.1 Instalacion y Pre-cacheo	14
10.2 Activacion y Limpieza de Cache	15
10.3 Estrategia de Fetch Hibrida	15
11. Autenticacion y Seguridad (JWT)	15
11.1 Flujo de Autenticacion	16
11.2 Middleware de Verificacion	16
11.3 Roles y Permisos	16
12. Notificaciones Push (Web Push)	16
12.1 Ciclo de Vida de una Notificacion	17
12.2 Tipos de Notificaciones	17
13. Comunicacion en Tiempo Real (Socket.IO)	17

13.1 Sistema de Rooms	17
13.2 Eventos del Sistema	18
14. API - Referencia de Endpoints	18
14.1 Acciones del Proxy Principal	19
15. Integracion con Google Apps Script	20
15.1 Modelo de Datos	20
15.2 Comunicacion con GAS	20
16. Instalacion y Configuracion	20
16.1 Requisitos Previos	20
16.2 Variables de Entorno	21
16.3 Pasos de Instalacion	21
17. Despliegue en Produccion	22
17.1 Backend en Railway	22
17.2 Frontend Estatico	22
18. Guia de Mantenimiento y Troubleshooting	22
18.1 Actualizacion de Version de Cache	23
18.2 Problemas Comunes y Soluciones	23
18.3 Recomendaciones de Escalabilidad	23

1. Introduccion y Vision General

DOMIDELIS es una Aplicacion Web Progresiva (PWA) disenada para gestionar el ciclo completo de domicilios de comida rapida, bebidas y otros productos locales en el municipio de Santuario, Colombia. La plataforma conecta a clientes, establecimientos (tiendas), domiciliarios y administradores en un ecosistema unificado que permite realizar pedidos, gestionar inventarios, asignar entregas y controlar finanzas desde cualquier dispositivo con navegador web.

El proyecto nace con la vision de digitalizar el proceso de domicilios en localidades pequenas y medianas, donde las soluciones existentes como Rappi o iFood no tienen presencia o resultan economicamente inviables para los comerciantes. DOMIDELIS ofrece una alternativa ligera, instalable directamente desde el navegador, que funciona sin conexion a internet para funciones basicas y que se integra con Google Sheets como base de datos a traves de Google Apps Script, eliminando la necesidad de servidores de base de datos dedicados.

La aplicacion esta compuesta por cuatro modulos diferenciados: el portal del cliente (donde se navegan tiendas, se arman carritos de compra y se confirman pedidos via WhatsApp), el panel de administracion (para gestionar tiendas, productos, domiciliarios, pedidos y generar informes financieros), el panel del domiciliario (con seguimiento de entregas en tiempo real, historial y notificaciones push) y la aplicacion de tiendas (un sub-sistema PWA independiente donde cada establecimiento gestiona sus productos y pedidos de forma aislada).

1.1 Caracteristicas Principales

A continuacion se describen las funcionalidades mas relevantes que conforman la plataforma DOMIDELIS, organizadas por modulo funcional para facilitar su comprension:

- **PWA instalable:** Los usuarios pueden instalar la aplicacion en sus dispositivos moviles o de escritorio directamente desde el navegador, obteniendo acceso desde el icono de la pantalla de inicio con experiencia similar a una aplicacion nativa.
- **Funcionamiento offline:** El Service Worker implementa una estrategia de caché inteligente que permite visualizar tiendas, productos previamente cargados y un juego interactivo cuando no hay conexion.
- **Notificaciones Push:** Mediante Web Push API con claves VAPID, los administradores y domiciliarios reciben alertas instantaneas ante nuevos pedidos o asignaciones, incluso con la aplicacion cerrada.
- **Tiempo real con Socket.IO:** Las actualizaciones de estado de pedidos, asignaciones de domiciliarios y notificaciones se propagan en tiempo real a todos los clientes conectados.

- **Autenticacion JWT por roles:** Sistema de tres roles (admin, domiciliario, tienda) con tokens JWT de 8 horas de vigencia, proteccion de rutas y verificacion en cada peticion al backend.
- **Informes financieros:** Modulo completo de informes con graficos Chart.js, calculo de comisiones, desglose por tienda y por domiciliario, filtros por fecha y metodos de pago, y exportacion CSV.
- **Carrito de compras multi-tienda:** Los clientes pueden agregar productos de diferentes establecimientos, calcular subtotales por tienda, seleccionar zona de envio con precios diferenciados y enviar el pedido por WhatsApp.
- **Banner de instalacion inteligente:** Detecta el navegador (Chrome, Safari, Samsung Internet, Firefox) y muestra instrucciones especificas para cada plataforma, incluyendo guias para iOS.
- **Anuncios promocionales:** Sistema de popup emergente con codigo promocional, validacion y almacenamiento para aplicacion automatica al finalizar la compra.
- **Juego offline integrado:** "Moto Runner", un juego tipo endless runner que se activa automaticamente cuando no hay conexion, con puntuacion y record local.

2. Arquitectura del Sistema

DOMIDELIS emplea una arquitectura de tres capas con un proxy intermedio que desacopla el frontend del almacenamiento de datos. El flujo de datos sigue un patron request-response clasico complementado con comunicacion bidireccional via WebSockets para funcionalidades en tiempo real. La persistencia de datos se delega a Google Apps Script (GAS) que actua como API REST sobre hojas de calculo de Google Sheets, eliminando la necesidad de una base de datos tradicional.

2.1 Diagrama de Componentes

El sistema se compone de los siguientes bloques funcionales, cada uno con responsabilidades claramente delimitadas y comunicacion bien definida:

Componente	Tecnologia	Funcion Principal
Frontend Cliente	HTML/CSS/JS + PWA	Navegacion, carrito, checkout, pedidos
Frontend Admin	HTML/CSS/JS + Chart.js	Gestion de tiendas, productos, domiciliarios, informes
Frontend Domiciliario	HTML/CSS/JS + Socket.IO	Entregas, historial, notificaciones push
App Tiendas (PWA)	HTML/CSS/JS + SW propio	Gestion aislada de productos y pedidos por tienda

Backend Proxy	Node.js + Express 5	Proxy API, JWT auth, Socket.IO, Web Push
Base de Datos	Google Sheets via GAS	Almacenamiento persistente de datos del negocio
Service Worker	Workbox-style custom	Cache offline, fallback, estrategia hibrida

2.2 Flujo de Datos

El flujo de una peticion tipica sigue el siguiente recorrido: el cliente (navegador) realiza una peticion HTTP al backend Express, que valida el token JWT si es necesario, enruta la peticion al endpoint correspondiente y la reenvia al Google Apps Script mediante una peticion HTTP con los parametros de accion (action). GAS procesa la accion sobre la hoja de calculo correspondiente y devuelve los datos en formato JSON. El backend responde al cliente con los datos obtenidos y, adicionalmente, emite eventos Socket.IO y notificaciones push si la accion lo requiere (por ejemplo, un nuevo pedido o una asignacion de domiciliario).

Para las operaciones en tiempo real, el flujo es diferente: los clientes se conectan al servidor Socket.IO y se unen a "rooms" segun su rol (admin_room, domiciliario_ID). Cuando ocurre un evento como la creacion de un pedido, el servidor emite el evento a los rooms correspondientes, y los clientes conectados actualizan su interfaz inmediatamente sin necesidad de recargar la pagina.

2.3 Estrategia de Desacoplamiento

La decision de usar Google Apps Script como capa de datos en lugar de una base de datos convencional responde a multiples factores practicos: no requiere infraestructura adicional ni costos de hosting para la base de datos, permite a los administradores visualizar y editar datos directamente en una hoja de calculo familiar, y ofrece una API REST gratuita con alta disponibilidad. El backend actua como proxy y añade capas de seguridad (JWT), comunicacion en tiempo real (Socket.IO) y notificaciones (Web Push) que GAS no proporciona nativamente. Esta separacion permite escalar o reemplazar la capa de datos en el futuro sin afectar el frontend ni la logica de negocio del backend.

3. Stack Tecnologico

A continuacion se detalla cada tecnologia utilizada en el proyecto, su proposito especifico y la justificacion de su eleccion dentro de la arquitectura de DOMIDELIS:

Capa	Tecnologia	Version	Proposito
Frontend	HTML5 + CSS3 + JavaScript	ES6+	Interfaz de usuario responsiva sin frameworks

Frontend	PWA (Manifest + SW)	-	Instalabilidad y funcionamiento offline
Frontend	Font Awesome	6.4.0	Iconografia consistente
Frontend	Google Fonts (Poppins)	-	Tipografia principal de la UI
Frontend	Chart.js	4.4.0	Graficos para informes financieros
Frontend	Socket.IO Client	4.6.1	Comunicacion en tiempo real
Backend	Node.js	18+	Runtime del servidor
Backend	Express	5.2.1	Framework HTTP para API y proxy
Backend	Socket.IO	4.8.3	WebSockets para tiempo real
Backend	jsonwebtoken	9.0.3	Generacion y verificacion de tokens JWT
Backend	web-push	3.6.7	Envio de notificaciones push (VAPID)
Backend	axios	1.15.0	Cliente HTTP para proxy a GAS
Backend	cors	2.8.6	Configuracion de CORS
Backend	dotenv	17.4.1	Variables de entorno desde .env
Datos	Google Apps Script	-	API REST sobre Google Sheets
Datos	Google Sheets	-	Almacenamiento persistente del negocio
Deploy	Railway	-	Hosting del backend Node.js
Deploy	GitHub Pages / Static	-	Hosting del frontend estatico
CI/CD	GitHub Actions	-	Despliegue automatico (deploy-frontend.yml)

La eleccion deliberada de no utilizar frameworks frontend como React, Vue o Angular responde a la necesidad de mantener el proyecto ligero y facil de mantener para un equipo pequeno. Vanilla JavaScript permite un control total sobre el rendimiento, elimina la curva de aprendizaje de frameworks y reduce drasticamente el tamano de los bundles. El peso total del frontend (sin imagenes) es inferior a 200KB, lo que garantiza tiempos de carga rapidos incluso en conexiones 3G. Ademias, esta decision facilita la compatibilidad con Service Workers y la instalacion como PWA sin las complejidades que introducen los frameworks en el renderizado del lado del cliente.

4. Estructura del Proyecto

El repositorio esta organizado en tres directorios principales que corresponden a las tres capas de la aplicacion. Cada directorio contiene sus propios activos (CSS, JS, imagenes) y es independiente del resto, lo que permite desplegar cada componente por separado. A continuacion se describe la estructura de directorios con explicacion de los archivos clave:

4.1 Directorio Raiz

El directorio raiz contiene los archivos de configuracion global del proyecto, incluyendo el archivo `.gitignore` para exclusiones del control de versiones y el directorio `.github/workflows` con la configuracion de CI/CD para despliegue automatico del frontend.

4.2 Directorio frontend/

Este directorio contiene toda la interfaz de usuario del cliente, administrador y domiciliario. Es servido como archivos estaticos por Express en desarrollo o por un CDN/servidor estatico en produccion. Los archivos HTML principales son:

Archivo	Modulo	Descripcion
index.html	Cliente	Pagina principal con tiendas, carrito, zona selector, anuncio promo
checkout.html	Cliente	Formulario de datos del cliente, resumen de pedido, metodo de pago
confirmacion.html	Cliente	Confirmacion visual tras enviar pedido
login.html	Admin/Domi	Autenticacion para administradores y domiciliarios
admin.html	Admin	Panel completo de administracion con informes y graficos
domiciliario.html	Domiciliario	Panel de entregas con tabs y juego offline
terminos.html	Publico	Terminos y condiciones de uso y codigos promocionales
manifest.json	PWA	Manifest del frontend principal
service-worker.js	PWA	Service Worker del frontend principal

Dentro de `frontend/assets/` se encuentran los subdirectorios `css/` (6 hojas de estilo modulares: `styles.css`, `anuncio.css`, `informe.css`, `notificaciones.css`, `offline-game.css`, `resumen-tienda.css`), `img/` (iconos PWA en 8 resoluciones de 72x72 a 512x512, logo y favicon), `js/` (13 archivos JavaScript incluyendo `admin.js`, `client.js`, `auth.js`, `checkout.js`, `config.js`, `domiciliario.js`, `push-manager.js`, `notificaciones.js`, `auth-guard.js`, `informe-financiero.js`, `info-domi.js`, `offline-game.js`, `sw-register.js` y el componente reutilizable `toast.js`), y `sounds/` (`alerta.mp3` para notificaciones audibles).

4.3 Directorio app-tiendas/

Este es un sub-sistema PWA completamente independiente con su propio manifest, service worker y conjunto de activos. Su aislamiento garantiza que cada tienda solo puede ver y gestionar sus propios productos y pedidos. Los archivos principales son:

Archivo	Descripcion
index-tienda.html	Panel principal de la tienda con tabs de productos y pedidos
login-tienda.html	Autenticacion exclusiva para establecimientos
manifest-tienda.json	Manifest PWA independiente (scope relativo)
sw-tienda.js	Service Worker propio con cache aislada
assets/css/tiendas.css	Estilos especificos del panel de tienda
assets/js/config-tienda.js	Configuracion de API y zona para tiendas
assets/js/tiendas.js	Logica de negocio del panel de tienda
assets/img/	Iconos PWA y logo de tienda (8 resoluciones)

4.4 Directorio backend/

El backend es una aplicacion Node.js con Express 5 que funciona como proxy API, servidor de WebSockets (Socket.IO), gestor de autenticacion JWT y motor de notificaciones push. Su estructura interna es la siguiente:

Archivo/Directorio	Descripcion
server.js	Punto de entrada principal: Express, Socket.IO, proxy, JWT, push
routes/tiendaRoutes.js	Router dedicado para endpoints de tiendas (/api/tienda/*)
middleware/verifyTienda.js	Middleware JWT exclusivo para rol tienda
package.json	Dependencias: express 5.2.1, socket.io 4.8.3, jsonwebtoken 9.0.3, etc.
.env	Variables de entorno: JWT_SECRET, VAPID keys, GAS_URL
railway.json	Configuracion de despliegue en Railway

5. Modulo Frontend - Cliente

El modulo del cliente es la cara visible de DOMIDELIS. Se accede a traves de index.html y ofrece una experiencia fluida desde la seleccion de la zona de envio hasta la confirmacion del pedido. Esta disenado con un enfoque mobile-first, priorizando la usabilidad en dispositivos moviles donde la mayoria de los usuarios realizan sus pedidos. La interfaz utiliza la tipografia Poppins de Google Fonts, una paleta de colores vibrantes (rojo primario #E63946, naranja secundario #F4A261, verde azulado de acento #2A9D8F) y componentes redondeados con sombras suaves que refuerzan la identidad visual de la marca.

5.1 Pagina Principal (index.html)

La pagina principal integra multiples funcionalidades clave que trabajan en conjunto para ofrecer una experiencia de compra completa. El hero section presenta un badge de lanzamiento animado, un descuento destacado del 50% y el selector de zona de envio con 16 zonas diferenciadas por precio. Debajo se muestra el grid de tiendas cargadas dinamicamente desde la API, con imagenes, descripciones, ratings y badges de categoria.

El carrito de compras es un componente flotante con panel lateral deslizable que permite agregar productos de multiples tiendas, ajustar cantidades, eliminar items y ver el total con el domicilio incluido. Al agregar el primer producto se ejecuta una animacion de "explosion de comida" con emojis que vuelan desde el boton hacia el carrito. El boton de checkout redirige a checkout.html con los datos del carrito almacenados en localStorage.

El sistema de anuncios promocionales muestra un popup emergente con codigo de descuento validable. El codigo se almacena en localStorage y se aplica automaticamente al finalizar la compra. El popup respeta la frecuencia de muestra (no aparece en cada visita) y puede cerrarse manualmente. Ademas, el banner de instalacion inteligente detecta el navegador del usuario y muestra instrucciones especificas para instalar la PWA, incluyendo guias visuales paso a paso para iOS (Safari), Android (Chrome, Samsung Internet, Firefox) y escritorio (Chrome).

5.2 Checkout y Confirmacion

La pagina de checkout (checkout.html) presenta un formulario con validaciones estrictas: nombre (max 50 caracteres), telefono (exactamente 10 digitos), direccion (max 60 caracteres), zona de envio (16 opciones con precios predefinidos) y referencias adicionales (max 100 caracteres). El resumen del pedido muestra los items agrupados por tienda con subtotales individuales, el valor del envio segun la zona seleccionada y el total final. Los metodos de pago disponibles son Efectivo, Nequi, Daviplata y Transferencia. Al confirmar, el pedido se envia al backend via API y se redirige a la pagina de confirmacion.

La pagina de confirmacion (confirmacion.html) muestra un mensaje de exito con la informacion del pedido, instrucciones sobre la confirmacion por WhatsApp (10-15 minutos) y enlaces directos para contactar via WhatsApp o volver a realizar otro pedido. El diseno es responsive y se adapta a pantallas moviles con botones de ancho completo para facilitar la interaccion tactil.

6. Modulo Frontend - Panel de Administracion

El panel de administracion (admin.html) es el centro de control de la plataforma. Solo es accesible para usuarios con rol "admin" autenticados mediante JWT. La interfaz utiliza un sistema de navegacion horizontal con tabs que organizan las seis secciones principales: Tiendas, Pedidos Activos, Domiciliarios, Historial, Informes y Pagos Domiciliarios. Cada seccion carga sus datos de forma diferida, es decir, solo cuando el administrador navega a ella por primera vez, optimizando asi el rendimiento de carga inicial.

6.1 Gestion de Tiendas y Productos

La seccion de Tiendas permite crear, editar y eliminar establecimientos con campos para nombre, descripcion, direccion, horario, rating, URL de imagen y porcentaje de comision para la app. Cada tienda tiene asociado un usuario de tipo "tienda" que puede acceder al sub-sistema app-tiendas. Los productos se gestionan mediante un selector de tienda que carga la lista de productos correspondientes, con campos para nombre, descripcion, precio, imagen y badge o etiqueta promocional. Las operaciones CRUD se realizan a traves del proxy API hacia GAS.

6.2 Pedidos Activos y Asignacion

La seccion de Pedidos Activos muestra en tiempo real todos los pedidos que no han sido entregados, actualizandose automaticamente mediante Socket.IO cuando llega un nuevo pedido. Cada pedido muestra ID, cliente, tienda, total, estado y domiciliario asignado. El administrador puede ver el detalle completo del pedido en un modal y asignar un domiciliario mediante un buscador con filtrado en tiempo real. La asignacion dispara un evento Socket.IO al room del domiciliario y una notificacion push simultanea.

6.3 Informes Financieros

El modulo de informes (informe-financiero.js) es una de las funcionalidades mas sofisticadas de la plataforma. Genera un dashboard completo con: 4 tarjetas KPI (Ingresos App, Egresos, Balance, Pedidos), diagrama de flujo de dinero (Cliente > App > Retencion + Devolucion), graficos de barras y donut con Chart.js (Ingresos vs Egresos por tienda, Distribucion de ventas, Metodos de pago), tabla de desglose por tienda con comisiones y pagos por metodo, y tienda destacada que mas vende. Los filtros permiten acotar por fecha, tienda, metodo de pago y periodos predefinidos (hoy, ayer, esta semana, este mes).

El modulo de Pagos Domiciliarios (info-domi.js) replica la estructura de informes pero enfocado en los pagos a domiciliarios: muestra cuanto cobro la app por domicilios, que comision retiene, cuanto se paga a cada domiciliario y graficas comparativas. Incluye

un diagrama de flujo de caja específico para domicilios y una tabla desglosada por domiciliario con el total de entregas y montos a pagar.

7. Modulo Frontend - Panel del Domiciliario

El panel del domiciliario (`domiciliario.html`) está diseñado para uso intensivo en movimiento, con una interfaz simplificada que prioriza la información accionable. Se accede solo con rol "domiciliario" y muestra dos tabs principales: Pedidos Activos (con badge contador en tiempo real) e Historial de Entregas (con estadísticas de ganancias y filtros por periodo). La conexión Socket.IO se establece automáticamente al cargar, uniéndolo al domiciliario a su room exclusivo para recibir asignaciones instantáneas.

Cada tarjeta de pedido muestra la información esencial para la entrega: ID, nombre del cliente, dirección, teléfono, productos, total y método de pago. Los botones de cambio de estado (pendiente > en camino > entregado) actualizan el pedido vía API y emiten eventos Socket.IO que se propagan al panel de administración en tiempo real. Cuando un pedido se marca como entregado, se envía una notificación push automática al administrador.

El historial muestra el total de entregas realizadas y las ganancias estimadas, con filtros por periodo (hoy, ayer, esta semana). La sección incluye un juego offline ("Moto Runner") que se activa automáticamente cuando se pierde la conexión, proporcionando entretenimiento al domiciliario mientras espera asignaciones. El juego utiliza un canvas HTML5 con renderizado pixelado, sistema de puntuación y almacenamiento de record en `localStorage`.

8. Modulo App-Tiendas (PWA Independiente)

La aplicación de tiendas es una PWA completamente aislada del frontend principal, con su propio manifest (`manifest-tienda.json`), service worker (`sw-tienda.js`) y cache independiente (`tienda-cache-v1`). Este diseño garantiza que las tiendas solo puedan acceder a sus propios datos y que la instalación como PWA sea independiente de la aplicación del cliente. El scope del manifest es relativo (`"."`), lo que permite que funcione correctamente tanto en desarrollo local como en producción con cualquier ruta base.

8.1 Autenticación de Tiendas

Las tiendas se autentican mediante un flujo dedicado en `/api/tienda/login`, que genera un token JWT con el rol "tienda" y lo almacena en `localStorage` como `"tienda_token"`. El middleware `verifyTienda.js` verifica que el token sea válido y que el rol sea exactamente "tienda", rechazando tokens de admin o domiciliario. Las rutas protegidas del router `tiendaRoutes.js` aplican este middleware a los endpoints de productos y pedidos,

asegurando que cada tienda solo pueda consultar sus propios datos filtrados por tiendaId.

8.2 Panel de Productos y Pedidos

El panel principal (index-tienda.html) tiene dos tabs: Productos (solo lectura, con aviso de que la edicion requiere plan Premium) y Pedidos. Los pedidos que ve la tienda son filtrados en el backend: de la lista completa de productos del pedido, solo se muestran los que pertenecen a esa tienda, y el total se recalcula como el subtotal exclusivo de sus productos. Esto se logra en tiendaRoutes.js parseando el campo productosJson, filtrando por tiendaId y sumando los subtotales. El resultado es que cada tienda ve solo su parte del pedido, sin acceso a informacion de otras tiendas o del domicilio.

8.3 Service Worker de Tiendas

El sw-tienda.js implementa la misma estrategia hibrida que el SW principal pero con cache y lista de archivos estaticos propios. Utiliza Promise.allSettled en la fase de instalacion para que un archivo faltante no impida la activacion del SW. En la fase de fetch, aplica Network First para llamadas API (con fallback a cache o respuesta JSON de error 503) y Cache First para archivos estaticos (con fallback a index-tienda.html para paginas HTML).

9. Backend - Servidor Node.js

El servidor backend (server.js) es el componente central de la infraestructura de DOMIDELIS. Implementa cinco funciones criticas en un unico archivo: servidor de archivos estaticos (solo en desarrollo), proxy API hacia Google Apps Script, servidor Socket.IO para comunicacion en tiempo real, motor de notificaciones push con Web Push API, y sistema de autentificacion JWT con proteccion de rutas por rol. A pesar de estar en un solo archivo, el codigo esta organizado en secciones claramente delimitadas con comentarios que facilitan su mantenimiento.

9.1 Configuracion del Servidor

El servidor utiliza Express 5.2.1 con un servidor HTTP subyacente que Socket.IO necesita para funcionar. La configuracion CORS es flexible: en desarrollo permite cualquier origen (origin: true), mientras que en produccion se deberia restringir a los dominios de la aplicacion. El middleware body-parser esta habilitado para JSON y URL-encoded, con el header Authorization permitido explicitamente en las configuraciones CORS de Express y Socket.IO para soportar la autentificacion JWT.

9.2 Proxy API Principal

El endpoint principal `/api` maneja todas las operaciones del negocio mediante un parametro "action" que determina la operacion a realizar. El flujo es: recibir peticion, verificar token JWT si existe, verificar permisos por rol segun la accion, construir la URL hacia GAS con los parametros, ejecutar la peticion, responder al cliente inmediatamente, y luego emitir eventos Socket.IO y notificaciones push si la accion lo requiere (post-response processing). Las acciones protegidas por rol se organizan en tres categorias: solo admin (crearTienda, actualizarTienda, eliminarTienda, crearProducto, actualizarProducto, eliminarProducto, crearDomiciliario, actualizarDomiciliario, eliminarDomiciliario, eliminarPedidos, asignarDomiciliario), admin o domiciliario (actualizarEstado), y cualquier usuario autenticado (getDomiciliarios, getPedidos).

9.3 Router de Tiendas

Las rutas de tiendas (`/api/tienda/*`) estan en un router Express separado (tiendaRoutes.js) que se monta en `/api/tienda`. Este router tiene tres endpoints: POST `/login` (autenticacion de tienda), GET `/productos` (productos de la tienda autenticada) y GET `/pedidos` (pedidos filtrados con calculo de subtotal). El middleware `verifyTienda` integrado en el router verifica el token JWT y el rol, inyectando los datos de la tienda en `req.tienda` para uso posterior.

9.4 Gestion de Suscripciones Push

El sistema de notificaciones push utiliza un Map en memoria para almacenar las suscripciones, donde la clave es el endpoint de la suscripcion y el valor contiene la suscripcion completa, el ID del usuario, el rol y la fecha de registro. Los endpoints disponibles son: POST `/api/suscripciones` (registrar nueva suscripcion), POST `/api/suscripciones/eliminar` (eliminar por endpoint), GET `/api/vapid-public-key` (obtener clave publica VAPID), POST `/api/enviar-push` (enviar notificacion a roles especificos) y GET `/api/suscripciones/estado` (ver suscripciones activas). Las suscripciones caducadas (HTTP 410 o 404) se eliminan automaticamente del Map.

10. Service Worker y Estrategia de Cache

DOMIDELIS implementa Service Workers personalizados (sin Workbox) con una estrategia de cache hibrida que combina Network First y Cache First segun el tipo de recurso. Este enfoque garantiza que los datos de la API esten siempre actualizados cuando hay conexion, mientras que los archivos estaticos se sirvan instantaneamente desde la cache. A continuacion se describe en detalle cada fase del ciclo de vida del Service Worker y la logica de cada estrategia.

10.1 Instalacion y Pre-cacheo

Durante la instalacion, el Service Worker precachea una lista de archivos estaticos esenciales que garantizan el funcionamiento basico offline. El precacheo utiliza `Promise.allSettled` en lugar de `Promise.all` para que un archivo faltante no impida la activacion. Los archivos cacheados incluyen: la pagina principal, el login, el manifest, el CSS, los archivos JS principales y los iconos PWA de 192x192 y 512x512. Tras el cacheo, se llama a `self.skipWaiting()` para activar el SW inmediatamente sin esperar a que se cierren las pestanas anteriores.

10.2 Activacion y Limpieza de Cache

En la fase de activacion, el Service Worker elimina todas las caches cuyo nombre comience con el prefijo correspondiente pero no coincidan con la version actual. Por ejemplo, si la version actual es "domidelis-cache-v2", se eliminarian "domidelis-cache-v1" y cualesquiera otras versiones anteriores. Despues de la limpieza, se llama a `self.clients.claim()` para que el SW tome control inmediato de todos los clientes abiertos, sin necesidad de recargar la pagina.

10.3 Estrategia de Fetch Hibrida

Tipo de Recurso	Estrategia	Fallback Offline
API (/api/)	Network First	Cache o JSON {error: "Sin conexion"} con status 503
HTML	Cache First	index.html desde cache
CSS, JS, Imagenes	Cache First	Recurso de cache o respuesta 408
POST/PUT/DELETE	Solo Red	No interceptados por el SW

La estrategia Network First para la API funciona asi: se intenta la peticion a la red primero; si tiene exito, se clona la respuesta y se guarda en cache para usos futuros; si la red falla, se busca en cache y, si no esta, se devuelve una respuesta JSON de error 503 en lugar de un `TypeError`. La estrategia Cache First para estaticos funciona al reves: se busca en cache primero y, si no se encuentra, se va a la red y se cachea la respuesta para la proxima vez. Si tanto la red como la cache fallan, se devuelve el `index.html` cacheado como fallback para paginas HTML, o una respuesta vacia con status 408 para otros recursos.

11. Autenticacion y Seguridad (JWT)

El sistema de autenticacion de DOMIDELIS utiliza JSON Web Tokens (JWT) con una estrategia híbrida que permite tanto acceso publico (clientes sin sesion) como acceso protegido por roles (admin, domiciliario, tienda). El secreto JWT se almacena en la variable de entorno `JWT_SECRET` y es verificado al iniciar el servidor, abortando el

proceso si no esta definido.

11.1 Flujo de Autenticacion

El flujo de login funciona en tres pasos: primero, el usuario envia sus credenciales (nombre y password) al endpoint POST /api/login o POST /api/tienda/login; segundo, el backend reenvia estas credenciales a Google Apps Script para validacion; tercero, si GAS confirma la identidad, el backend genera un token JWT con el payload {id, nombre, rol} y una expiracion de 8 horas. El token se devuelve al cliente, que lo almacena en localStorage y lo envia en el header Authorization: Bearer en cada peticion posterior.

11.2 Middleware de Verificacion

El middleware verificarToken (en server.js) implementa una logica híbrida: si no hay token, permite continuar con req.user = null (acceso publico); si hay token pero es invalido o expirado, devuelve 403 (esto permite al frontend expulsar al usuario); si el token es valido, inyecta los datos decodificados en req.user. El middleware verifyTienda (en routes/tiendaRoutes.js) es mas estricto: requiere token y rol "tienda", devolviendo 401 si falta el token, 403 si el rol no coincide, y 400 si el token es invalido. Esta diferencia de comportamiento permite que los clientes no autenticados naveguen tiendas y productos, mientras que las operaciones sensibles requieren autenticacion estricta.

11.3 Roles y Permisos

Rol	Acceso	Endpoints Protegidos
admin	Panel admin + todas las operaciones CRUD	crearTienda, eliminarTienda, crearProducto, etc.
domiciliario	Panel domiciliario + actualizar estado	actualizarEstado, getPedidos, getDomiciliarios
tienda	App-tiendas (solo sus datos)	/api/tienda/productos, /api/tienda/pedidos
publico (sin token)	Navegacion y pedidos de cliente	crearPedido, getTiendas, getProductos

12. Notificaciones Push (Web Push)

El sistema de notificaciones push utiliza la Web Push API con el protocolo VAPID (Voluntary Application Server Identification), que permite enviar notificaciones a los usuarios sin necesidad de un servicio de terceros. Las claves VAPID se generan una unica vez y se almacenan en el archivo .env del backend. La clave publica se expone al

frontend a traves del endpoint GET /api/vapid-public-key para que los clientes puedan suscribirse.

12.1 Ciclo de Vida de una Notificacion

El ciclo completo de una notificacion push funciona de la siguiente manera: primero, el cliente solicita permiso al usuario mediante `Notification.requestPermission()`; segundo, si el usuario acepta, se crea una suscripcion `PushSubscription` usando `registration.pushManager.subscribe()` con la clave publica VAPID; tercero, la suscripcion se envia al backend via POST /api/suscripciones junto con el ID y rol del usuario; cuarto, cuando ocurre un evento que requiere notificacion (nuevo pedido, asignacion, entrega), el backend itera sobre las suscripciones del rol correspondiente y envia la notificacion mediante `webpush.sendNotification()`; quinto, el Service Worker del cliente recibe el evento "push" y muestra la notificacion nativa del sistema operativo. Las suscripciones que devuelven HTTP 410 (Gone) o 404 (Not Found) se eliminan automaticamente del Map de suscripciones.

12.2 Tipos de Notificaciones

Tipo	Trigger	Destinatario	Datos del Payload
Nuevo pedido	crearPedido	admin	titulo, mensaje, pedidoId, icon
Asignacion	asignarDomiciliario	domiciliario especifico	titulo, cuerpo con cliente y total
Pedido entregado	actualizarEstado = entregado	admin	titulo, mensaje, pedidoId

13. Comunicacion en Tiempo Real (Socket.IO)

Socket.IO proporciona la capa de comunicacion bidireccional que permite que los cambios en el estado de los pedidos se reflejen instantaneamente en todos los paneles conectados, sin necesidad de polling o recarga manual. El servidor Socket.IO se configura con CORS permisivo en desarrollo y soporta los headers de autenticacion para futuras validaciones de sockets.

13.1 Sistema de Rooms

Los clientes se identifican al conectarse mediante el evento "identificar" que envia su rol e ID. Segun el rol, se unen a rooms especificos: los domiciliarios se unen a "domiciliario_ID" (por ejemplo, domiciliario_5), lo que permite enviar mensajes dirigidos a un domiciliario especifico sin que los demas lo reciban; los administradores se unen a "admin_room", un room compartido donde se notifican todos los eventos

relevantes. Este sistema de rooms es fundamental para la funcionalidad de asignacion de pedidos, donde el servidor emite el evento "nuevoPedidoAsignado" exclusivamente al room del domiciliario asignado.

13.2 Eventos del Sistema

Evento	Direccion	Descripcion
identificar	Cliente > Servidor	Registra rol e ID, une a room correspondiente
nuevoPedido	Servidor > Todos	Notifica nuevo pedido al panel admin
estadoActualizado	Servidor > Todos	Propaga cambio de estado de pedido
nuevoPedidoAsignado	Servidor > Domiciliario	Notifica asignacion a domiciliario especifico
pedidoAsignado	Servidor > Todos	Confirma asignacion a todos los paneles
disconnect	Cliente > Servidor	Desconexion del socket

Los eventos se emiten en el bloque post-response del proxy API, es decir, despues de responder al cliente con los datos de GAS. Este diseno garantiza que la respuesta HTTP no se retrase por la emision de eventos Socket.IO o notificaciones push. Los errores en la emision de eventos se capturan y registran en consola sin afectar la respuesta al cliente.

14. API - Referencia de Endpoints

A continuacion se documenta cada endpoint disponible en el backend de DOMIDELIS, incluyendo el metodo HTTP, la ruta, los parametros requeridos, la autentificacion necesaria y una descripcion funcional. Todos los endpoints de datos retornan JSON.

Metodo	Ruta	Auth	Descripcion
POST	/api/login	No	Login general (admin/domiciliario), retorna JWT
POST	/api/tienda/login	No	Login exclusivo de tiendas, retorna JWT con rol tienda
GET/P OST	/api?action=...	Segun accion	Proxy principal hacia GAS
GET	/api/tienda/productos	Tienda JWT	Productos de la tienda autenticada
GET	/api/tienda/pedidos	Tienda JWT	Pedidos filtrados por tiendaId
GET	/api/vapid-public-key	No	Clave publica VAPID para suscripcion push

POST	/api/suscripciones	No	Registrar suscripcion push
POST	/api/suscripciones/eliminar	No	Eliminar suscripcion push por endpoint
POST	/api/enviar-push	No	Enviar notificacion a roles especificos
GET	/api/suscripciones/estado	No	Listar suscripciones activas
GET	/api/status	No	Estado del servidor (online/offline, modo)

14.1 Acciones del Proxy Principal

El endpoint /api acepta un parametro "action" que determina la operacion a realizar en GAS. Las acciones disponibles y su nivel de proteccion son las siguientes:

Accion	Rol Requerido	Descripcion
login	Ninguno	Validar credenciales contra GAS
getTiendas	Ninguno	Obtener lista de tiendas activas
getProductos	Ninguno	Obtener productos (por tiendaId opcional)
crearPedido	Ninguno	Registrar nuevo pedido con datos del cliente
getPedidos	Autenticado	Obtener pedidos (filtrados por rol)
getDomiciliarios	Autenticado	Lista de domiciliarios registrados
crearTienda	Admin	Crear nuevo establecimiento
actualizarTienda	Admin	Actualizar datos de una tienda
eliminarTienda	Admin	Eliminar una tienda por ID
crearProducto	Admin	Crear producto asociado a una tienda
actualizarProducto	Admin	Modificar datos de un producto
eliminarProducto	Admin	Eliminar un producto por ID
crearDomiciliario	Admin	Registrar nuevo domiciliario
actualizarDomiciliario	Admin	Modificar datos de domiciliario
eliminarDomiciliario	Admin	Eliminar domiciliario por ID
asignarDomiciliario	Admin	Asignar domiciliario a un pedido
actualizarEstado	Admin/Domi	Cambiar estado de un pedido
eliminarPedidos	Admin	Eliminar pedidos por IDs

15. Integracion con Google Apps Script

Google Apps Script (GAS) funciona como la capa de persistencia de DOMIDELIS, exponiendo una API REST publica que el backend Node.js consume como proxy. Los datos se almacenan en hojas de calculo de Google Sheets, donde cada hoja representa una entidad del modelo de datos (tiendas, productos, pedidos, domiciliarios, usuarios). Esta aproximacion tiene ventajas significativas para un proyecto de esta escala: costo cero de infraestructura de base de datos, interfaz visual directa para administradores, API REST automatica con alta disponibilidad, y la posibilidad de editar datos manualmente en la hoja de calculo como mecanismo de respaldo.

15.1 Modelo de Datos

El modelo de datos se distribuye en hojas de calculo separadas dentro del mismo spreadsheet. Cada fila representa un registro y la primera fila contiene los encabezados. Los campos criticos del modelo son: Tiendas (id, nombre, password, direccion, imagen, comision), Productos (id, tiendaId, nombre, descripcion, precio, imagen, badge), Pedidos (id, clienteNombre, clienteTelefono, direccion, zona, productosJson, total, estado, domiciliarioId, metodoPago, fecha), Domiciliarios (id, nombre, telefono, password, comisionApp), y Usuarios (id, nombre, password, rol). El campo productosJson almacena un array JSON con los productos del pedido, incluyendo tiendaId para filtrar por tienda.

15.2 Comunicacion con GAS

La comunicacion se realiza mediante peticiones HTTP GET o POST al URL del Web App de GAS, pasando la accion como parametro de consulta y los datos adicionales como parametros de consulta (GET) o como form-urlencoded (POST). GAS responde con JSON que contiene al menos un campo "success" booleano. El backend no almacena datos de forma persistente; toda la persistencia reside en GAS. La unica excepcion es el Map de suscripciones push, que se almacena en memoria y se pierde al reiniciar el servidor, lo cual es aceptable dado que los clientes se resuscriben automaticamente al cargar la aplicacion.

16. Instalacion y Configuracion

Esta seccion describe los pasos necesarios para configurar el entorno de desarrollo, instalar las dependencias y ejecutar el proyecto localmente. Se detallan los requisitos previos, las variables de entorno obligatorias y los comandos de inicio para cada componente.

16.1 Requisitos Previos

Requisito	Version Minima	Proposito
Node.js	18.x o superior	Runtime del backend
npm	9.x o superior	Gestion de dependencias
Git	2.x	Control de versiones
Navegador moderno	Chrome 80+ / Safari 14+	Testing de PWA y Service Workers
Cuenta Google	-	Acceso a Google Sheets y Apps Script
ngrok (opcional)	-	Tunel HTTPS para testing en moviles

16.2 Variables de Entorno

El archivo `.env` en el directorio `backend/` contiene todas las variables de entorno necesarias para el funcionamiento del servidor. A continuacion se listan las variables obligatorias con su descripcion y formato esperado:

Variable	Requerida	Descripcion
JWT_SECRET	Si	Clave secreta para firmar tokens JWT (minimo 32 caracteres)
VAPID_PUBLIC_KEY	Si	Clave publica VAPID para Web Push (generada con web-push)
VAPID_PRIVATE_KEY	Si	Clave privada VAPID para Web Push (no compartir)
VAPID_EMAIL	Si	Email de contacto para VAPID (mailto:correo@ejemplo.com)
PORT	No	Puerto del servidor (default: 80)
NODE_ENV	No	production o development (default: development)

Para generar las claves VAPID, ejecute el comando: `npx web-push generate-vapid-keys`. Este comando producira una clave publica y una clave privada que deben copiarse en el archivo `.env`. La clave publica debe ser la misma que se utiliza en el frontend (`config.js`) para que las suscripciones push sean compatibles con el servidor.

16.3 Pasos de Instalacion

El proceso de instalacion completo sigue estos pasos secuenciales. Primero, clonar el repositorio: `git clone`. Segundo, instalar dependencias del backend: `cd backend && npm install`. Tercero, crear el archivo `.env` con las variables descritas anteriormente. Cuarto, iniciar el servidor: `npm start`. Quinto, verificar que el servidor este corriendo accediendo a `http://localhost:80/api/status`. Sexto, para probar en dispositivos moviles, usar ngrok: `ngrok http 80` y acceder a la URL proporcionada.

En modo desarrollo, Express sirve el frontend y app-tiendas como archivos estaticos desde los directorios ../frontend y ../app-tiendas respectivamente. En produccion, el frontend se despliega independientemente (por ejemplo, en GitHub Pages o un CDN) y el backend solo expone la API. La variable `NODE_ENV=production` desactiva el servicio de archivos estaticos.

17. Despliegue en Produccion

DOMIDELIS utiliza un modelo de despliegue separado para frontend y backend, lo que permite actualizar cada componente de forma independiente y optimizar costos de infraestructura. El backend se despliega en Railway (plataforma PaaS para Node.js), mientras que el frontend se sirve como archivos estaticos desde un CDN o servidor web.

17.1 Backend en Railway

Railway es la plataforma de despliegue elegida para el backend debido a su simplicidad de configuracion, soporte nativo para Node.js y variables de entorno, y su plan gratuito que es suficiente para el volumen de trafico esperado. El archivo `railway.json` contiene la configuracion de despliegue. Las variables de entorno se configuran directamente en el dashboard de Railway, incluyendo `JWT_SECRET`, `VAPID_PUBLIC_KEY`, `VAPID_PRIVATE_KEY` y `VAPID_EMAIL`. Railway detecta automaticamente el comando de inicio (`npm start`) desde `package.json` y asigna un puerto via la variable `PORT`. El despliegue se realiza automaticamente al hacer push a la rama principal del repositorio.

17.2 Frontend Estatico

El frontend se despliega como archivos estaticos en un servidor web o CDN. El workflow de GitHub Actions (`.github/workflows/deploy-frontend.yml`) automatiza el despliegue al hacer push a la rama principal. Los unicos cambios necesarios para produccion son: actualizar la variable `API_URL` en `config.js` para que apunte a la URL del backend en Railway (en lugar de `localhost`), y actualizar el `scope` y `start_url` del `manifest.json` si la aplicacion se sirve desde un subdirectorio. El archivo `.vscode/settings.json` puede contener configuraciones del editor para el equipo de desarrollo.

18. Guia de Mantenimiento y Troubleshooting

Esta seccion proporciona guias para el mantenimiento continuo de la plataforma, incluyendo la actualizacion de la version de cache, la resolucion de problemas comunes y las recomendaciones para el crecimiento del proyecto.

18.1 Actualizacion de Version de Cache

Cada vez que se modifican archivos estaticos del frontend, es necesario actualizar la version de la cache en el Service Worker para que los usuarios reciban los cambios. El proceso es: cambiar el nombre de la cache (por ejemplo, de "domidelis-cache-v1" a "domidelis-cache-v2") en service-worker.js, actualizar la lista ARCHIVOS_ESTATICOS si se anadieron o eliminaron archivos, y hacer deploy. Cuando el nuevo SW se active, eliminara automaticamente la cache anterior y precacheara los archivos actualizados. Los usuarios con la pestana abierta recibiran la actualizacion al recargar la pagina; los que no la tengan abierta la recibiran en la proxima visita. Se recomienda versionar la cache de forma incremental para facilitar el rastreo de problemas.

18.2 Problemas Comunes y Soluciones

Problema	Causa Probable	Solucion
SW no se actualiza	Cache vieja en el navegador	DevTools > Application > Clear storage > Unregister SW
Push no llega	Suscripcion expirada (410)	El sistema la elimina auto; el cliente se resuscribe al cargar
Token JWT expirado	Sesion > 8 horas	El usuario debe volver a iniciar sesion
GAS devuelve error	Hoja de calculo modificada	Verificar estructura de hojas y permisos del script
Offline no funciona	SW no registrado o cache vacia	Verificar registro del SW en consola y recargar
Carrito vacio al recargar	localStorage limpiado	Datos persisten en localStorage; verificar modo incognito
Socket.IO no conecta	CORS o firewall	Verificar configuracion CORS y puertos abiertos

18.3 Recomendaciones de Escalabilidad

A medida que DOMIDELIS crezca en usuarios y volumen de pedidos, se recomienda considerar las siguientes mejoras arquitectonicas: migrar la capa de datos de Google Sheets a una base de datos real (PostgreSQL o MongoDB) para mejorar rendimiento y consistencia; implementar una cola de mensajes (Redis o RabbitMQ) para gestionar notificaciones push de forma mas robusta; agregar persistencia a las suscripciones push (actualmente en memoria) usando una base de datos o Redis; implementar rate limiting en los endpoints de la API para prevenir abuso; agregar tests unitarios y de integracion automatizados; y considerar una refactorizacion del frontend a un framework componentizado (como Vue.js o React) cuando la complejidad lo justifique.

Adicionalmente, se recomienda monitorear el rendimiento del backend con herramientas como PM2 o New Relic, implementar logging estructurado (Winston o Pino) en lugar de console.log, y configurar alertas automaticas para caidas del servidor o errores de conexion con GAS. La separacion actual en archivos estaticos facilita la migracion gradual: se puede empezar por reemplazar la capa de datos sin tocar el frontend, y luego ir migrando los modulos del frontend uno por uno segun prioridad.